IP ASSIGNMENT - 2
Rubric

Q1

Yardstick	Marks	Rationale
Code runs	2	Code is running on the students' system without any error.
Explainability	1	Student is able to explain the working of the code
File Handling	2	File handling with proper read and write operations being done.
Deliverables	5	• Menu-driven(1) • Display Specific Word Count(1) • Display Unique Words(1) • Display All Word Count(1) • Replace Word(1)

The two outputs shown below, should be producible on the students' system. Otherwise, marks would be deducted for the "code runs" component.

```
--------------------------------------------------------------------------------
Enter your choice:
1. Display specific Word Count
2. Display all Unique Words
3. Display all Word Counts
4. Replace word
5. Quit
1
Enter word: the
Word Count:  12


--------------------------------------------------------------------------------
Enter your choice:
1. Display specific Word Count
2. Display all Unique Words
3. Display all Word Counts
4. Replace word
5. Quit
1
Enter word: .
Word Count:  22
```

```
--------------------------------------------------------------------------------
Enter your choice:
1. Display specific Word Count
2. Display all Unique Words
3. Display all Word Counts
4. Replace word
5. Quit
4
Enter word to be replaced: the
Enter word that will replace the: counts
Replaced successfully!


--------------------------------------------------------------------------------
Enter your choice:
1. Display specific Word Count
2. Display all Unique Words
3. Display all Word Counts
4. Replace word
5. Quit
1
Enter word: counts
Word Count:  12
```

Q2

Yardstick	Marks	Rationale

Code runs	2	Code is running on the students' system (1) and the queries are correctly inputting (1)
Explainability	2	Student is able to explain the implementation(2)
Test Case	1	The test case provided below is giving correct output
Deliverables	5	• Function for matrix multiplication (2) • All three types of queries are implemented (1+1+1)

→See if appropriate functions are made.
→A single matrix multiplication function is made for reuse purposes. Give 1 mark out of 2 if matrix multiplication code is written again and again in different functions.
→ Some students might have different implementations for matrix multiplication, but that should be valid and generic.

Test Case (You can make your own test case if want to):

Input:
9
3 -1 -1 3 3 -1 -1 6
1 1 1 1 5 5 5 5 6
-2 -2 4 4 4 -2 -2 4 6
3
S 2 1.5 3
T 1 2 3
R z 0.785   ##--> 0.785 is basically $\pi/4$: some students might have implemented in degrees.

Output:
Each successive lines denote the respective space-separated x, y, and z coordinates
for all the points after transformation
Transformed X's

```
2.47487373 -3.18198052 -3.18198052  2.47487373 -1.76776695 -1.76776695
 -7.4246212  -7.4246212   1.41421356
```

Transformed Y's

```
7.4246212   1.76776695  1.76776695  7.4246212  11.66726189 11.66726189
  6.01040764  6.01040764 16.97056275
```

Transformed Z's

```
-3.0 -3.0 15.0 15.0 15.0 -3.0 -3.0 15.0 21.0
```

Some students might get another row with all ones, that is acceptable. The answers may vary in terms of decimal precision.

→Values closer to expected output are also acceptable.
→**phi(ϕ) in degrees/radians both is acceptable**

Q3

Yardstick	Marks	Rationale
Code runs	2	Code is running on the students' system(1) and is Menu driven(1)
Explainability	2	Student is able to explain the implementation(2)
Deliverables	6	• Functions for each of the 7 tasks (3) • Task 1 to 6, both ways conversion has been implemented (2) • Task 7, user input for A and B (1)

→If a generic function has been designed by the student instead of 7 different functions, and if does work as expected, full marks will be awarded for that component (i.e, 3 marks).

→ Negative or fractional values are not required. If somebody has implemented it. Well done!!
→**The TAs would provide any random value for any or all of the components, and the code output must show the expected value. Otherwise, 2 marks would be deducted from the "code runs" component.**

Q4

Yardstick	Marks	Rationale

Code runs	2	Code is running on the students' system without any error.
Explainability	1	Student is able to explain the working of the code
File Handling	2	File handling with proper read and write operations being done. And the FinalReport txt file is created.
Deliverables	5	• Correctly reading the student files (2) • Correctly marking the students(1) • Correctly writing the FinalReport file (2)

Q5

Yardstick	Marks	Rationale
Code runs	2	Code runs without error on the students' system.
Explainability	2	Student is able to explain the problem statement.
Deliverables	6	• Function noteCreate() → (2) • 'scaleMajor.txt' and 'scaleMinor.txt' created and written inside noteCreate()→ (2) • Functions majorNotes() and minorNotes() implemented → (2)

majorNotes() and minorNotes() must be reading the files 'scaleMajor.txt' and 'scaleMinor.txt' respectively.
The notes can be written to the files using the rules given, either by implementing any algorithm or simply hard-coding and then writing it to the files.

The output of this code can vary as some of the students would have used higher octave notes (i.e, with an apostrophe)... If anyone has implemented some other type of scale (blues, jazz, gypsy, Carnatic, or Hindustani classical), he will also be given marks given he/she/they have, to some extent, implemented the deliverables correctly. Scope for improvisation, allows the scales, to break the rules as well!

Possible solution (can vary, depending upon whether a student has used multiple apostrophes or missed some notes)

```
Enter the root note: F
Enter the type of scale (Major/Minor): Minor
The Minor scale in the key of F is:
F G G# A# C C# D# F'
```

Q6

Yardstick	Marks	Rationale
Code runs	2	Code runs without any error on the students' system.
Explainability	2	Student is able to explain his implementation.
Deliverables	6	Code should be able to perform each of the 6 tasks (6X1)

If the code is not generic, i.e, as mentioned in the question, does not handle any arbitrary NXN matrix, where the matrix is itself given by the user, then 2 marks would be deducted from the **deliverables.**
The TAs would provide any random matrix, and the result should be similar to the expectations, otherwise, 2 marks would be deducted from the "code runs " component.

B1

This problem is implementation-dependent from the students' perspective. Check the logic of the implemented functions [function1 and function2] is correct or not. The rest of the components such as file handling is mandatory.

Yardstick	Marks	Rationale
Code runs	2	The code runs on the students' system without any

		error
Explainability	2	The student is able to explain his implementation.
Deliverables	6	Cases.py(1) ● function1→ (1) ● generateData→(1) test.py(1) ● function2 → (1) ● testing → (1)

function1() and function2() depends upon the choice of the student. The naming itself could be different.
File handling operations to create and read input/output pairs in generateData() and testing() must have been implemented. **Otherwise**, 1 mark would be deducted.
Cases.py must be called inside test.py. Otherwise, 2 marks would be deducted.

B2

Yardstick	Marks	Rationale
Code runs	3	The code runs on the students' system without any error
Explainability	2	The student is able to explain his implementation.
Deliverables	5	● Correct implementations to calculate the height of skyscraper (2) ● Optimal choosing of skyscrapers(1) ● Correct expected output (1)